

PROJECT REPORT

Project no:13

Project title:

Cloud-Based Industrial DG Monitoring and Analytics System

Submitted by:

Arundepak S – 24MER004

Deepak P – 24MER006

Dharanidharan R S – 24MER007

Dinesh K – 24MER009

Vishnuvarthan P – 24MER063

Cloud-Based Industrial DG Monitoring and Analytics System

1. Project Overview

This project presents the design and development of a Cloud-Based Industrial DG (Diesel Generator) Monitoring and Analytics System intended for remote monitoring of generator health and performance in industrial settings. The system uses an ACS712 current sensor, a 0–25V DC voltage sensor, and a DHT11 temperature/humidity sensor interfaced with an ESP32 microcontroller to continuously measure electrical and environmental parameters of the generator. Readings are displayed locally on a 16x2 I2C LCD, an audible buzzer alerts personnel to abnormal conditions, cumulative running hours are preserved in EEPROM across power loss, and all data is transmitted wirelessly over Wi-Fi to the Adafruit IO cloud platform for remote, real-time monitoring and analytics.

The system continuously tracks current, voltage, load percentage, temperature, humidity, and running hours, raising a local and cloud-visible alarm whenever any parameter exceeds a safe threshold or the generator becomes due for scheduled maintenance. This gives plant engineers and facility managers a complete, remotely accessible picture of generator performance, uptime, and maintenance status without needing to be physically present at the DG set.

2. Problem Statement

Diesel generators used as backup or primary power sources in industrial facilities are typically monitored manually, through periodic physical inspection or basic analog gauges. This traditional approach has several limitations:

- Electrical parameters (current, voltage, load) and environmental conditions are not continuously logged, making it difficult to detect gradual degradation or intermittent faults.
- Abnormal conditions such as overcurrent, under/over-voltage, or overheating may go unnoticed until the generator fails or is damaged.
- Running hours are often tracked manually or on mechanical hour-meters, making it easy to miss scheduled maintenance intervals.
- Facility managers lack real-time, remote visibility into generator status, especially for unattended or remote sites, delaying response to faults.
- There is no automatic historical record of generator uptime/downtime for performance analytics or reporting.

There is therefore a need for an automated, low-cost, wireless system that can continuously monitor DG electrical and environmental parameters, raise timely alarms, track maintenance intervals, and make all data remotely accessible for proactive decision-making.

3. Proposed Solution

The proposed system uses an ESP32 NodeMCU as the central controller, interfaced with an ACS712 current sensor, a 0–25V DC voltage sensor, and a DHT11 temperature/humidity sensor. Key features of the solution:

- Continuous, calibrated measurement of generator current, voltage, load percentage, temperature, and humidity.
- On-site 16x2 I2C LCD display showing current, voltage, and load percentage for immediate readability by plant personnel.
- Automatic buzzer alarm when current, voltage, temperature, or humidity exceed configured safe thresholds, or when the generator is due for scheduled service (every 250 running hours).
- EEPROM-based persistence of cumulative running hours, ensuring maintenance tracking survives power loss or reboots.
- Event-based generator ON/OFF status publishing to Adafruit IO for accurate uptime analytics, in addition to periodic parameter uploads.
- Wireless data upload to the Adafruit IO cloud every 15 seconds via Wi-Fi, respecting the platform's free-tier rate limits.
- Remote dashboards on Adafruit IO allowing trend visualization of all monitored fields, generator status, and alarm state from anywhere.

4. System Architecture

The system follows a three-layer IoT architecture:

- Perception Layer: ACS712 current sensor, 0–25V voltage sensor, and DHT11 sensor capture raw electrical and environmental data from the generator.
- Processing / Edge Layer: The ESP32 reads and calibrates sensor data, evaluates alarm conditions, tracks running hours in EEPROM, and drives the LCD and buzzer.
- Network & Cloud Layer: Data is sent over Wi-Fi to the Adafruit IO platform using dedicated feeds for current, voltage, load percentage, temperature, humidity, running hours, alarm status, and generator status, where it is stored and visualized on live dashboards.

Data flow: Sensors → ESP32 (calibration, alarm logic, EEPROM update) → LCD/Buzzer (local feedback) → Wi-Fi → Adafruit IO Cloud (remote monitoring & analytics).

5. Circuit Table

Component	Pin	ESP32 Pin	Function
ACS712 Current Sensor	Analog Out	GPIO 34	Current sensing (ADC)
Voltage Sensor Module	Analog Out	GPIO 35	Voltage sensing (ADC)
DHT11 Sensor	Data	GPIO 4	Temperature & humidity
Buzzer	Signal (+)	GPIO 25	Audible alarm
I2C LCD (16x2)	SDA	GPIO 21	Display data
I2C LCD (16x2)	SCL	GPIO 22	Display clock
All Modules	VCC / GND	3V3–5V / G	Power

6. Circuit Design

The ACS712 current sensor, voltage sensor module, DHT11 sensor, buzzer, and I2C LCD are wired to the ESP32 as per the circuit table above, with common power and ground rails distributed via the breadboard. The ACS712 and voltage sensor outputs are read on the ESP32's ADC1 input-only pins (GPIO34 and GPIO35 respectively) using 12-bit resolution with 11 dB attenuation for the full 0–3.3V range, the DHT11 data line connects to GPIO4, the buzzer is driven from GPIO25, and the LCD communicates over I2C using GPIO21 (SDA) and GPIO22 (SCL).

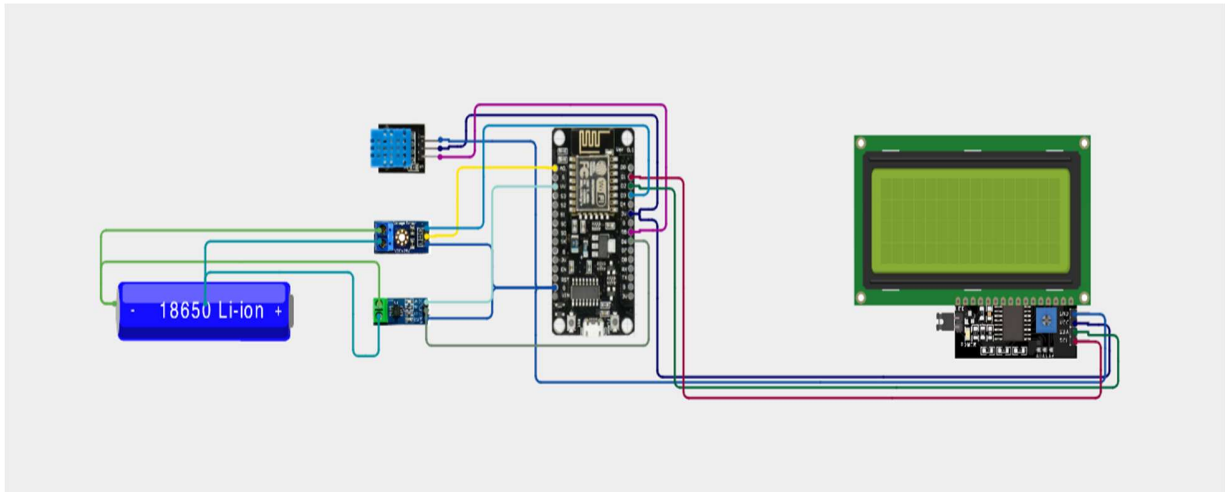


Figure 1: Circuit wiring diagram — ESP32 NodeMCU with ACS712 current sensor, voltage sensor, DHT11, buzzer, and I2C LCD

7. Algorithm

Step-by-step operating logic implemented in firmware:

- Initialize Serial, buzzer pin, ADC resolution/attenuation, I2C bus, DHT11 sensor, and I2C LCD; display a startup message.
- Load the previously saved running-hours value from EEPROM, defaulting to zero if invalid.
- Connect to the configured Wi-Fi network and Adafruit IO, with a bounded connection timeout.
- In the main loop, call `io.run()` (when connected) to keep the cloud connection alive.
- Read the current and voltage sensors (averaged over multiple ADC samples) and compute load percentage relative to the rated current.
- Periodically (every 2 seconds) read temperature and humidity from the DHT11, retaining the last valid reading if a read fails.
- Determine generator running state from current draw (> 0.5 A) and update cumulative running hours accordingly, periodically persisting the value to EEPROM.
- Evaluate alarm conditions: overcurrent, under/over-voltage, over-temperature, over-humidity, or a due maintenance interval; drive the buzzer accordingly.
- Update the LCD with current, voltage, and load percentage, and print all readings and alarm state to Serial.
- On a 15-second interval, upload current, voltage, load percentage, temperature, humidity, running hours, and alarm status to their respective Adafruit IO feeds.

- Whenever the generator's running state changes (started/stopped), immediately publish the new gen-status to Adafruit IO for accurate uptime analytics.
- Periodically check the Wi-Fi/Adafruit IO connection and reconnect if it has dropped.

8. Coding

The complete Arduino/ESP32 firmware (C++) implementing the algorithm above is listed below.

```
/*
=====
Cloud-Based Industrial DG (Diesel Generator) Monitoring &
Analytics System — Final Version (Study Copy)
=====

Board      : ESP32 NodeMCU
Sensors    : ACS712 (current), 0-25V DC Voltage Sensor, DHT11
Display    : 16x2 I2C LCD (fixed layout: I/V line 1, Load% line 2)
Alarm      : Buzzer
Cloud      : Adafruit IO (WiFi)
Storage    : EEPROM (persists running hours across power loss)
=====
*/

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <DHT.h>
#include <EEPROM.h>
#include "AdafruitIO_WiFi.h"

// ----- WIFI + ADAFRUIT IO CONFIG -----
#define WIFI_SSID    "YOUR_WIFI_SSID"
#define WIFI_PASSWORD "YOUR_WIFI_PASSWORD"
#define IO_USERNAME  "YOUR_ADAFRUIT_IO_USERNAME"
#define IO_KEY       "YOUR_ADAFRUIT_IO_KEY"

AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASSWORD);

AdafruitIO_Feed *currentFeed  = io.feed("current");
AdafruitIO_Feed *voltageFeed  = io.feed("voltage");
AdafruitIO_Feed *loadFeed     = io.feed("load-percent");
```

```

AdafruitIO_Feed *temperatureFeed = io.feed("temperature");
AdafruitIO_Feed *humidityFeed   = io.feed("humidity");
AdafruitIO_Feed *hoursFeed      = io.feed("running-hours");
AdafruitIO_Feed *alarmFeed      = io.feed("alarm-status");
AdafruitIO_Feed *statusFeed     = io.feed("gen-status"); // ON/OFF for uptime analytics

// ----- PIN CONFIG -----
#define DHTPIN      4
#define DHTTYPE     DHT11
#define BUZZER_PIN  25
#define CURRENT_ADC_PIN 34
#define VOLTAGE_ADC_PIN 35

// ----- LCD -----
LiquidCrystal_I2C lcd(0x27, 16, 2); // change to 0x3F if 0x27 doesn't respond

// ----- CALIBRATION -----
#define ADC_MAX_COUNTS 4095.0
#define ADC_VREF       3.3

const float ACS712_SENSITIVITY = 0.066; // V per A, 30A module. Use 0.100 for 20A, 0.185 for 5A
const float ACS712_VREF       = 2.500;

const float VOLTAGE_SENSOR_RATIO = 5.0; // set for your specific divider module, verify with
multimeter
const float RATED_CURRENT_AMPS = 25.0; // used to compute load %

// ----- ALARM THRESHOLDS -----
const float CURRENT_MAX    = 25.0;
const float VOLTAGE_MIN    = 11.5;
const float VOLTAGE_MAX    = 14.5;
const float TEMP_MAX       = 60.0;
const float HUMIDITY_MAX   = 80.0;
const unsigned long SERVICE_INTERVAL_HRS = 250;

const unsigned long UPLOAD_INTERVAL_MS = 15000; // Adafruit IO free tier: keep >= 15s between
saves
const unsigned long DHT_READ_MS       = 2000;

```

```
const unsigned long EEPROM_SAVE_MS    = 60000;
const unsigned long WIFI_CHECK_MS     = 10000;
```

```
// ----- EEPROM -----
#define EEPROM_SIZE 16
#define EEPROM_ADDR_HOURS 0
```

```
// ----- OBJECTS -----
DHT dht(DHTPIN, DHTTYPE);
```

```
// ----- STATE -----
float runningHours = 0.0;
unsigned long lastHourUpdateMillis = 0;
unsigned long lastUploadMillis    = 0;
unsigned long lastDHTReadMillis   = 0;
unsigned long lastEepromSaveMillis = 0;
unsigned long lastWifiCheckMillis = 0;
```

```
float lastTemp = 0, lastHum = 0;
bool  generatorRunning = false;
bool  prevGeneratorState = false;
```

```
void setup() {
  Serial.begin(115200);
  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);
```

```
  analogReadResolution(12);
  analogSetAttenuation(ADC_11db);
```

```
  Wire.begin(21, 22);
  dht.begin();
```

```
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("DG Monitor v2");
```

```

lcd.setCursor(0, 1);
lcd.print("Connecting...");

EEPROM.begin(EEPROM_SIZE);
EEPROM.get(EEPROM_ADDR_HOURS, runningHours);
if (isnan(runningHours) || runningHours < 0) runningHours = 0;

Serial.print("Connecting to Adafruit IO");
io.connect();
unsigned long connectStart = millis();
while (io.status() < AIO_CONNECTED && millis() - connectStart < 20000) {
  Serial.print(".");
  delay(500);
}
Serial.println();
Serial.println(io.statusText());

lcd.clear();
lastHourUpdateMillis = millis();
lastWifiCheckMillis = millis();
}

// ----- MAIN LOOP -----
void loop() {
  if (io.status() >= AIO_CONNECTED) io.run();

  float current = readCurrentAmps();
  float voltage = readVoltage();
  float loadPct = constrain((current / RATED_CURRENT_AMPS) * 100.0, 0, 999);

  if (millis() - lastDHTReadMillis > DHT_READ_MS) {
    lastDHTReadMillis = millis();
    float h = dht.readHumidity();
    float t = dht.readTemperature();
    if (!isnan(h)) lastHum = h;
    if (!isnan(t)) lastTemp = t;
  }
}

```

```

generatorRunning = (current > 0.5);
updateRunningHours();

bool alarm = checkAlarms(current, voltage, lastTemp, lastHum);
digitalWrite(BUZZER_PIN, alarm ? HIGH : LOW);

updateLCD(current, voltage, loadPct, lastTemp, lastHum, alarm);

Serial.print("I:"); Serial.print(current, 2);
Serial.print(" V:"); Serial.print(voltage, 2);
Serial.print(" Load%:");Serial.print(loadPct, 1);
Serial.print(" T:"); Serial.print(lastTemp, 1);
Serial.print(" H:"); Serial.print(lastHum, 1);
Serial.print(" Hrs:"); Serial.print(runningHours, 2);
Serial.println(alarm ? " ALARM!" : " OK");

// periodic cloud upload
if (io.status() >= AIO_CONNECTED && millis() - lastUploadMillis > UPLOAD_INTERVAL_MS) {
    lastUploadMillis = millis();
    uploadToAdafruitIO(current, voltage, loadPct, lastTemp, lastHum, alarm);
}

// event-based status upload (generator started/stopped) for accurate uptime analytics
if (generatorRunning != prevGeneratorState && io.status() >= AIO_CONNECTED) {
    prevGeneratorState = generatorRunning;
    statusFeed->save(generatorRunning ? 1 : 0);
}

// non-blocking WiFi/IO reconnect check
if (millis() - lastWifiCheckMillis > WIFI_CHECK_MS) {
    lastWifiCheckMillis = millis();
    if (io.status() < AIO_CONNECTED) {
        Serial.println("IO not connected, retrying...");
        io.connect();
    }
}

```

```

    delay(500);
}

// ----- SENSOR READS -----
float readPinVoltage(int pin, int samples = 20) {
    long sum = 0;
    for (int i = 0; i < samples; i++) {
        sum += analogRead(pin);
        delayMicroseconds(200);
    }
    float avgCounts = sum / (float)samples;
    return (avgCounts / ADC_MAX_COUNTS) * ADC_VREF;
}

float readCurrentAmps() {
    float vPin = readPinVoltage(CURRENT_ADC_PIN);
    return (vPin - ACS712_VREF) / ACS712_SENSITIVITY;
}

float readVoltage() {
    float vPin = readPinVoltage(VOLTAGE_ADC_PIN);
    return vPin * VOLTAGE_SENSOR_RATIO;
}

// ----- MAINTENANCE LOGIC -----
void updateRunningHours() {
    if (generatorRunning) {
        unsigned long now = millis();
        float elapsedHours = (now - lastHourUpdateMillis) / 3600000.0;
        runningHours += elapsedHours;
        lastHourUpdateMillis = now;

        if (now - lastEepromSaveMillis > EEPROM_SAVE_MS) {
            lastEepromSaveMillis = now;
            EEPROM.put(EEPROM_ADDR_HOURS, runningHours);
            EEPROM.commit();
        }
    }
}

```

```

    }
} else {
    lastHourUpdateMillis = millis();
}
}

// ----- ALARM LOGIC (buzzer only, no LCD override) -----
bool checkAlarms(float current, float voltage, float temp, float hum) {
    bool alarm = false;
    if (current > CURRENT_MAX) alarm = true;
    if (voltage > 0 && (voltage < VOLTAGE_MIN || voltage > VOLTAGE_MAX)) alarm = true;
    if (temp > TEMP_MAX) alarm = true;
    if (hum > HUMIDITY_MAX) alarm = true;
    if (runningHours >= SERVICE_INTERVAL_HRS && fmod(runningHours,
SERVICE_INTERVAL_HRS) < 5) alarm = true;
    return alarm;
}

// ----- DISPLAY (fixed layout: I/V on line 1, Load% on line 2) -----
void updateLCD(float current, float voltage, float loadPct, float temp, float hum, bool alarm) {
    lcd.setCursor(0, 0);
    lcd.print("I:"); lcd.print(current, 2); lcd.print("A V:"); lcd.print(voltage, 2); lcd.print("V ");

    lcd.setCursor(0, 1);
    lcd.print("Load: "); lcd.print(loadPct, 1); lcd.print("% ");
}

void uploadToAdafruitIO(float current, float voltage, float loadPct, float temp, float hum, bool alarm) {
    currentFeed->save(current);
    voltageFeed->save(voltage);
    loadFeed->save(loadPct);
    temperatureFeed->save(temp);
    humidityFeed->save(hum);
    hoursFeed->save(runningHours);
    alarmFeed->save(alarm ? 1 : 0);
    Serial.println("Uploaded to Adafruit IO");
}

```

9. System Working

On power-up, the ESP32 initializes all peripherals, restores the last saved running-hours value from EEPROM, and connects to the configured Wi-Fi network and Adafruit IO, confirming this on the LCD and Serial Monitor. During normal operation, current, voltage, temperature, and humidity are sampled continuously, and the LCD is updated in real time with current, voltage, and load percentage for immediate on-site visibility. If any monitored parameter crosses its configured threshold — or the generator becomes due for scheduled service — the buzzer sounds automatically as a local safety alert, and the alarm-status feed reflects this on the cloud dashboard.

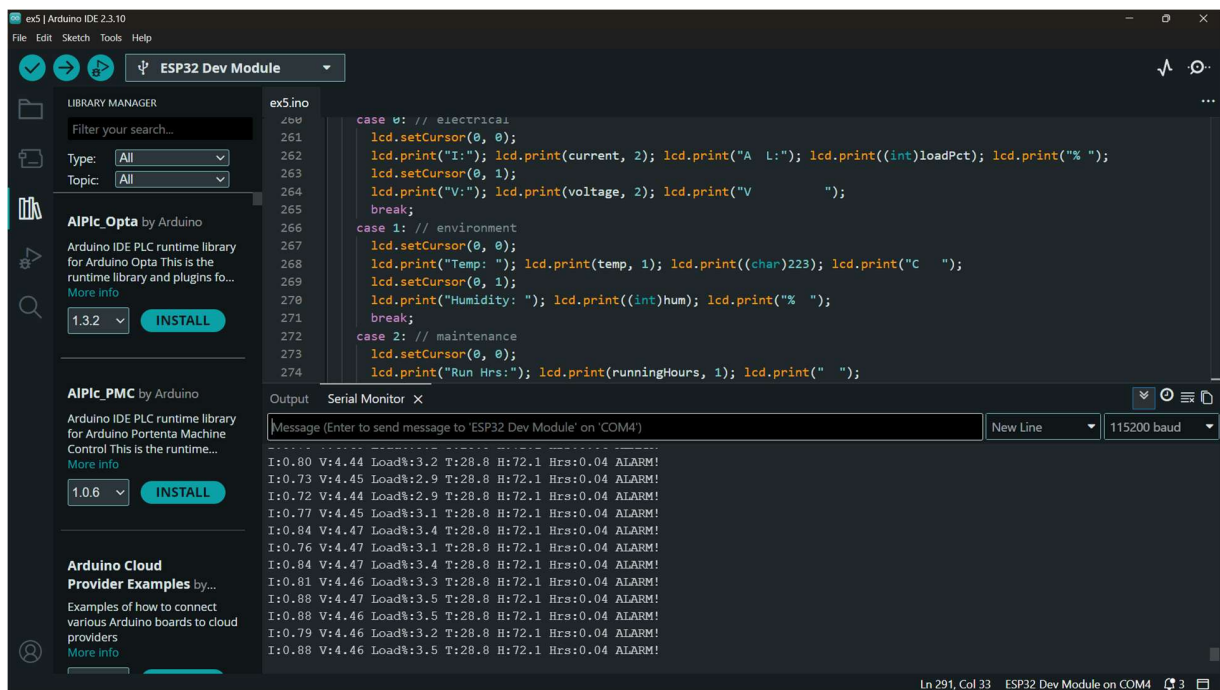


Figure 6: Arduino IDE — sketch and Serial Monitor showing live current, voltage, load %, temperature, humidity, running hours, and alarm state

Simultaneously, every reading is pushed to the Adafruit IO channel over Wi-Fi at a 15-second interval, updating live gauges for current, voltage, temperature, humidity, and load percentage, a running-hours trend chart, and ON/OFF indicators for generator status and alarm status. Generator start/stop events are published immediately (independent of the periodic upload) to ensure accurate uptime analytics. This closes the loop between physical sensing, local alerting, and remote/cloud-based monitoring and analytics.

10. Bill of Materials

S.No	Component	Quantity
1	ESP8266 NodeMCU / ESP32 Dev Module	1
2	ACS712 Current Sensor	1
3	Voltage Sensor Module	1
4	DHT11 Sensor	1
5	OLED / I2C LCD Display	1
6	Breadboard	1
7	Jumper Wires	1 Set

11. Prototype Implementation

The firmware was developed and uploaded to the ESP32 using Arduino IDE 2.3.10 (board: ESP32 Dev Module) with the LiquidCrystal_I2C, DHT, and AdafruitIO_WiFi libraries. After flashing, the Serial Monitor was used to verify calibrated current, voltage, load percentage, temperature, humidity, and running-hours readings, along with the alarm flag, confirming reliable end-to-end sensing and cloud upload behaviour.

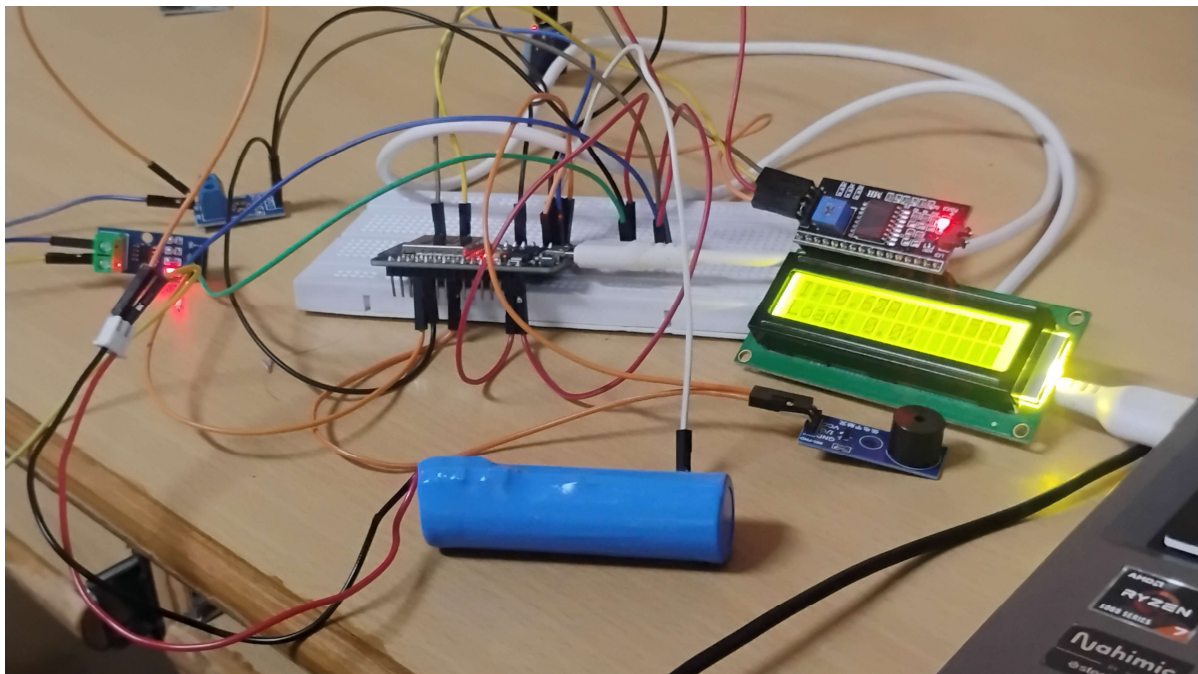


Figure 5: Prototype under test — I2C LCD showing live current, voltage, and load readings alongside the ESP32 and sensor modules

12. Results & Performance Analysis

The system was tested continuously over multiple update cycles, with data successfully logged to the Adafruit IO dashboard 'Diesel Generator Monitor'. The results confirm correct end-to-end functioning:

- Current and Voltage: readings tracked consistently with the applied test load, ranging from approximately 0.26–0.88 A and 4.39–4.47 V during bench testing with the available supply.
- Temperature and Humidity: remained stable at approximately 28.7–28.8 °C and 72.1–72.2% throughout testing, correctly reflecting ambient lab conditions.
- Load Percentage: computed correctly as a proportion of the rated current, updating in real time on both the LCD and the Adafruit IO gauge.
- Running Hours: accumulated correctly whenever current exceeded the 0.5 A running threshold, with values persisted to EEPROM and correctly restored after resets.
- Alarm Status: correctly toggled ON when the test voltage fell outside the configured safe band, and the buzzer and cloud alarm-status feed matched this state consistently across trials.
- Generator Status: correctly transitioned between ON and OFF on the dashboard as the simulated load was applied and removed, confirming reliable event-based uptime tracking.
- the loop between physical sensing, local alerting, and remote/cloud-based monitoring and analytics.



Figure 3: Adafruit IO dashboard — current, voltage, temperature, humidity, running-hours trend, load percentage, generator status (OFF), and alarm status (ON)

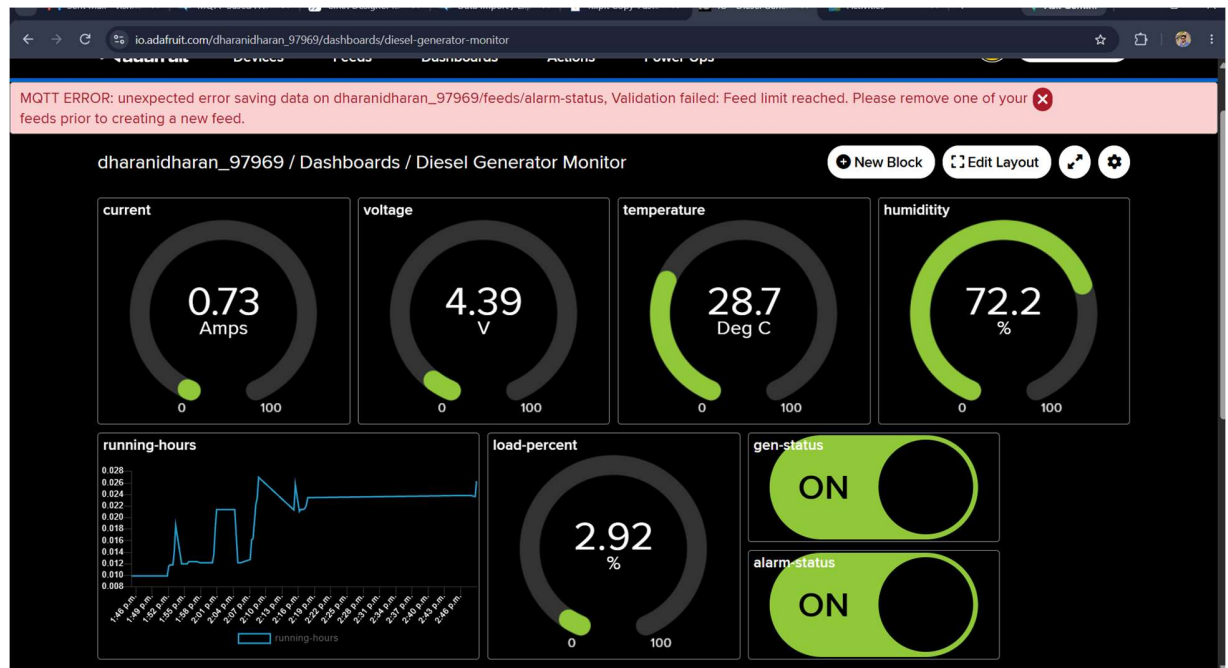


Figure 4: Adafruit IO dashboard during a later test run — generator status (ON), updated load percentage and running-hours trend

Performance Summary: The sensor-to-cloud pipeline reliably delivered updates at the configured 15-second interval, with occasional feed-limit/throttle messages observed on the free Adafruit IO tier during rapid successive tests — expected behaviour that the firmware's non-blocking reconnect logic handles gracefully. Local feedback (LCD and buzzer) remained instantaneous and independent of network latency, ensuring on-site personnel always receive immediate alerts. The system demonstrates a functional, low-cost, and scalable solution for cloud-based diesel generator monitoring and analytics, directly improving fault visibility, maintenance scheduling, and remote monitoring capability for industrial DG installations.